Rossmann Store Sales Forecasting

Step 1 — Import libraries and load data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.dummy import DummyRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error
from xgboost import XGBRegressor

sns.set_theme(style="whitegrid")

# Change these paths if your CSV files are stored elsewhere
train = pd.read_csv("/content/train.csv", parse_dates=["Date"])
store = pd.read_csv("/content/store.csv")

print("Train shape:", train.shape)
print("Store shape:", store.shape)
display(train.head())
display(store.head())
```

```
Train shape: (728579, 9)
Store shape: (1115, 10)
/tmp/ipykernel_614/3736265691.py:18: DtypeWarning: Columns (7) have mixed types. Specify dty
  train = pd.read_csv("/content/train.csv", parse_dates=["Date"])
```

	Store	DayOfWeek	Date	Sales	Customers	Open	Promo	StateHoliday	SchoolHoliday
0	1	5	2015-07-31	5263.0	555.0	1.0	1.0	0	1.0
1	2	5	2015-07-31	6064.0	625.0	1.0	1.0	0	1.0
2	3	5	2015-07-31	8314.0	821.0	1.0	1.0	0	1.0
3	4	5	2015-07-31	13995.0	1498.0	1.0	1.0	0	1.0
4	5	5	2015-07-31	4822.0	559.0	1.0	1.0	0	1.0

	Store	StoreType	Assortment	CompetitionDistance	CompetitionOpenSinceMonth	CompetitionO
0	1	c	a	1270.0	9.0	
1	2	a	a	570.0	11.0	
2	3	a	a	14130.0	12.0	

Step 2 — Understand and check the data

Check the columns, data types, missing values, duplicate records, and target summary. Do not delete columns just because they contain missing values; some missingness is meaningful.

```
print(train.info())
print(store.info())

train.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 728579 entries, 0 to 728578
Data columns (total 9 columns):
 #   Column         Non-Null Count   Dtype
---  ------         --------------   -----
 0   Store          728579 non-null  int64
 1   DayOfWeek      728579 non-null  int64
 2   Date           728579 non-null  object
 3   Sales          728578 non-null  float64
 4   Customers      728578 non-null  float64
 5   Open           728578 non-null  float64
 6   Promo          728578 non-null  float64
 7   StateHoliday   728578 non-null  object
 8   SchoolHoliday  728578 non-null  float64
dtypes: float64(5), int64(2), object(2)
memory usage: 50.0+ MB
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1115 entries, 0 to 1114
Data columns (total 10 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Store                      1115 non-null   int64
 1   StoreType                  1115 non-null   object
 2   Assortment                 1115 non-null   object
 3   CompetitionDistance        1112 non-null   float64
 4   CompetitionOpenSinceMonth  761 non-null    float64
 5   CompetitionOpenSinceYear   761 non-null    float64
 6   Promo2                     1115 non-null   int64
 7   Promo2SinceWeek            571 non-null    float64
 8   Promo2SinceYear            571 non-null    float64
 9   PromoInterval              571 non-null    object
dtypes: float64(5), int64(2), object(3)
memory usage: 87.2+ KB
None
```

	Store	DayOfWeek	Sales	Customers	Open	Promo	Schoo
count	728579.000000	728579.000000	728578.000000	728578.000000	728578.000000	728578.000000	7285
mean	558.498992	3.997046	5861.205509	637.598510	0.829480	0.379616	
std	321.965531	1.996620	3904.784060	467.347977	0.376089	0.485292	
min	1.000000	1.000000	0.000000	0.000000	0.000000	0.000000	
25%	280.000000	2.000000	3793.250000	410.000000	1.000000	0.000000	
50%	558.000000	4.000000	5834.000000	614.000000	1.000000	0.000000	
75%	838.000000	6.000000	7970.000000	842.000000	1.000000	1.000000	
max	1115.000000	7.000000	41551.000000	5494.000000	1.000000	1.000000	

```
store.describe()
```

	Store	CompetitionDistance	CompetitionOpenSinceMonth	CompetitionOpenSinceYear	
count	1115.00000	1112.000000	761.000000	761.000000	1
mean	558.00000	5404.901079	7.224704	2008.668857	
std	322.01708	7663.174720	3.212348	6.195983	
min	1.00000	20.000000	1.000000	1900.000000	
25%	279.50000	717.500000	4.000000	2006.000000	
50%	558.00000	2325.000000	8.000000	2010.000000	
75%	836.50000	6882.500000	10.000000	2013.000000	
max	1115.00000	75860.000000	12.000000	2015.000000	

Step 3 — Basic exploratory data analysis (EDA)

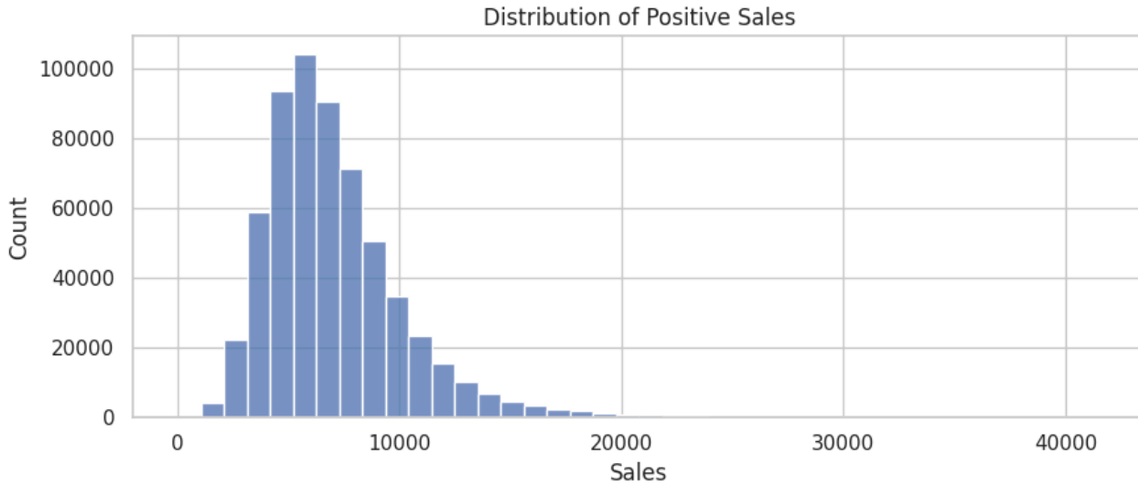
These charts help you understand sales patterns before training.

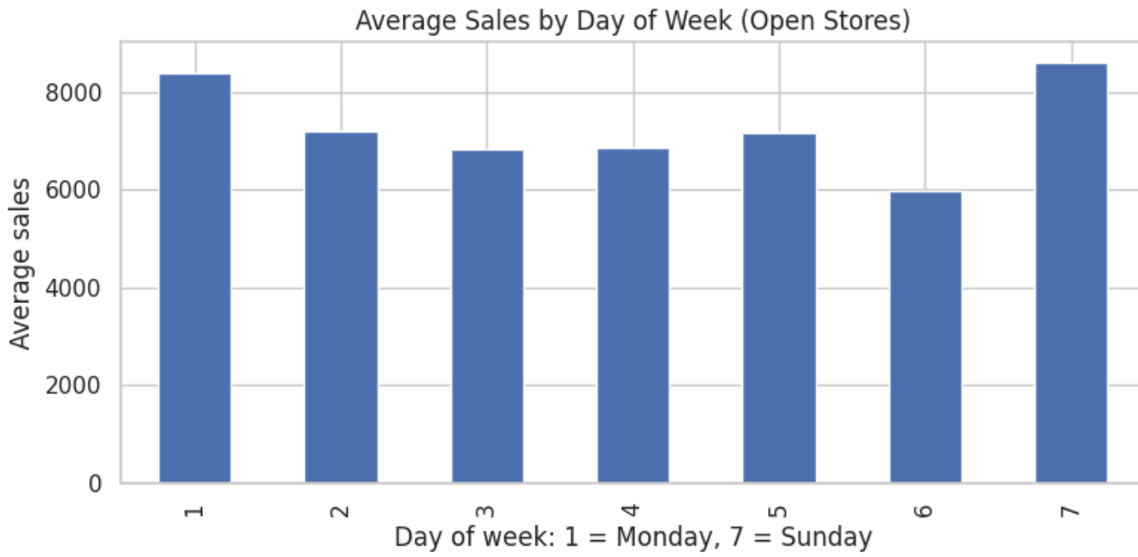
```
# Sales distribution, excluding zero-sales rows so the positive values are easier to inspe
plt.figure(figsize=(9, 4))
sns.histplot(data=train[train["Sales"] > 0], x="Sales", bins=40)
plt.title("Distribution of Positive Sales")
plt.tight_layout()
```

```
plt.show()

# Average sales by weekday for open stores
open_data = train[train["Open"] == 1].copy()
weekday_sales = open_data.groupby("DayOfWeek")["Sales"].mean()
weekday_sales.plot(kind="bar", figsize=(8, 4))
plt.title("Average Sales by Day of Week (Open Stores)")
plt.xlabel("Day of week: 1 = Monday, 7 = Sunday")
plt.ylabel("Average sales")
plt.tight_layout()
plt.show()

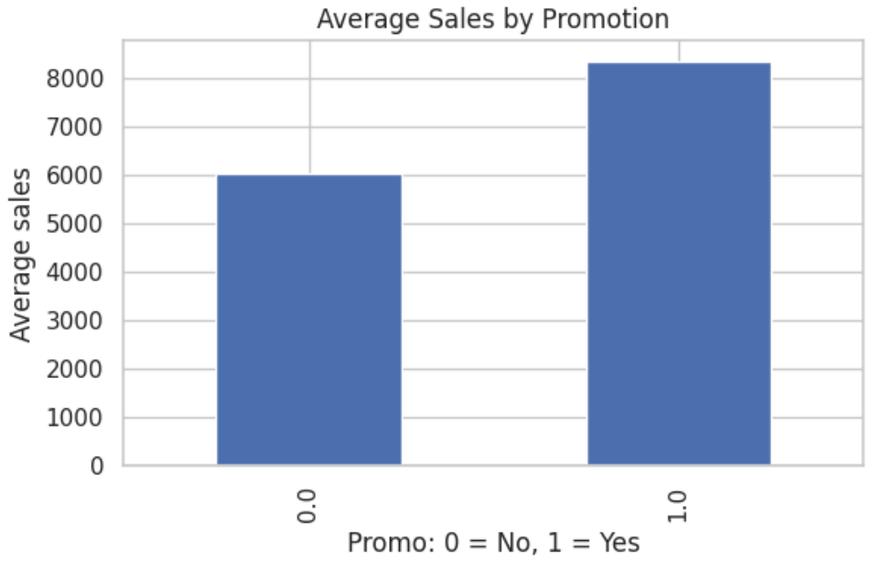
# Compare sales on promotion and non-promotion days
promo_sales = open_data.groupby("Promo")["Sales"].mean()
print("Average sales by promotion status:")
display(promo_sales.to_frame("average_sales"))
promo_sales.plot(kind="bar", figsize=(6, 4))
plt.title("Average Sales by Promotion")
plt.xlabel("Promo: 0 = No, 1 = Yes")
plt.ylabel("Average sales")
plt.tight_layout()
plt.show()
```





Average sales by promotion status:

Promo	average_sales
0.0	6033.606276
1.0	8354.564940



Step 4 — Join store information and create simple features

We merge the two files using `Store`. We predict sales for open stores, so closed-store rows are excluded from model training. We create year/month/day/week features from the date. `Customers` and raw `Date` are not used as model inputs.

```
data = train.merge(store, on="Store", how="left", validate="many_to_one")
data = data[data["Open"] == 1].copy()

data["Year"] = data["Date"].dt.year
```

```
data["Month"] = data["Date"].dt.month
data["Day"] = data["Date"].dt.day
data["WeekOfYear"] = data["Date"].dt.isocalendar().week.astype(int)
data["IsWeekend"] = data["Date"].dt.dayofweek.isin([5, 6]).astype(int)

# Treat StateHoliday as a category; use a consistent string format
data["StateHoliday"] = data["StateHoliday"].astype(str).replace({"0.0": "0"})

print("Rows after merging and filtering open stores:", data.shape)
display(data.head())
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_614/3735323598.py in <cell line: 0>()
      2 data = data[data["Open"] == 1].copy()
      3
----> 4 data["Year"] = data["Date"].dt.year
      5 data["Month"] = data["Date"].dt.month
      6 data["Day"] = data["Date"].dt.day

                                 2 frames
/usr/local/lib/python3.13/dist-packages/pandas/core/indexes/accessors.py in __new__(cls,
data)
    641             return PeriodProperties(data, orig)
    642
--> 643         raise AttributeError("Can only use .dt accessor with datetimelike values")

AttributeError: Can only use .dt accessor with datetimelike values
```

Next steps: Explain error

More EDA — sales trends and seasonality

These charts use open stores only. Weekly averages make the overall trend easier to see, while the heatmap shows how month and weekday interact.

```
# Mean sales per open store on each date, summarized by week
daily_mean_sales = data.groupby("Date")["Sales"].mean().sort_index()
weekly_mean_sales = daily_mean_sales.resample("W").mean()

fig, ax = plt.subplots(figsize=(13, 4))
ax.plot(weekly_mean_sales.index, weekly_mean_sales, alpha=0.55, label="Weekly average")
ax.plot(weekly_mean_sales.index, weekly_mean_sales.rolling(8, min_periods=1).mean(),
        linewidth=2.5, label="8-week moving average")
ax.set(title="Average Sales per Open Store Over Time", xlabel="Date", ylabel="Average sales
ax.legend()
plt.tight_layout()
plt.show()
```

```
# Average sales by month and weekday across the available dates
month_weekday_sales = data.pivot_table(
    index="DayOfWeek", columns="Month", values="Sales", aggfunc="mean"
)
fig, ax = plt.subplots(figsize=(13, 5))
sns.heatmap(month_weekday_sales, annot=True, fmt=".0f", cmap="YlGnBu",
            cbar_kws={"label": "Average sales"}, ax=ax)
ax.set(title="Average Sales by Month and Day of Week",
       xlabel="Month", ylabel="Day of week (1 = Monday)")
plt.tight_layout()
plt.show()
```

More EDA — differences between store types

The bar chart compares promotion days within each store type. The box plot shows the spread of sales, using a fixed random sample to keep plotting responsive.

```
promo_by_store_type = data.groupby(["StoreType", "Promo"])["Sales"].mean().unstack()
promo_by_store_type = promo_by_store_type.rename(
    columns={0: "No promotion", 1: "Promotion"}
)

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
promo_by_store_type.plot(kind="bar", ax=axes[0], rot=0)
axes[0].set(title="Average Sales by Store Type and Promotion",
            xlabel="Store type", ylabel="Average sales")
```

```
axes[0].legend(title="Promotion status")

store_sample = data.sample(n=min(20_000, len(data)), random_state=42)
sns.boxplot(data=store_sample, x="StoreType", y="Sales",
            showfliers=False, ax=axes[1])
axes[1].set(title="Sales Spread by Store Type (Sample)",
            xlabel="Store type", ylabel="Sales")
plt.tight_layout()
plt.show()
```

Step 5 — Choose input features (X) and target (y)

`X` contains information available when making the forecast. `y` is the actual sales value. We deliberately exclude `Customers` to avoid using information that would not normally be known for a future date.

```
target = "Sales"

categorical_features = [
    "Store", "DayOfWeek", "StateHoliday", "StoreType", "Assortment", "PromoInterval"
]
numeric_features = [
    "Promo", "SchoolHoliday", "CompetitionDistance",
    "CompetitionOpenSinceMonth", "CompetitionOpenSinceYear",
    "Promo2", "Promo2SinceWeek", "Promo2SinceYear",
    "Year", "Month", "Day", "WeekOfYear", "IsWeekend"
]

# Use only columns present in the merged data
categorical_features = [c for c in categorical_features if c in data.columns]
numeric_features = [c for c in numeric_features if c in data.columns]
feature_columns = categorical_features + numeric_features

X = data[feature_columns]
y = data[target]

print("Number of input features:", len(feature_columns))
print("Target:", target)
print("X shape:", X.shape, "| y shape:", y.shape)
```

Step 6 — Split by time

For forecasting, train on earlier dates and validate on later dates. This better simulates predicting the future than randomly splitting rows.

```
validation_days = 42
cutoff_date = data["Date"].max() - pd.Timedelta(days=validation_days - 1)

train_data = data[data["Date"] < cutoff_date].copy()
valid_data = data[data["Date"] >= cutoff_date].copy()

X_train = train_data[feature_columns]
y_train = train_data[target]
X_valid = valid_data[feature_columns]
y_valid = valid_data[target]

print("Training dates:", train_data["Date"].min().date(), "to", train_data["Date"].max().da
print("Validation dates:", valid_data["Date"].min().date(), "to", valid_data["Date"].max().
print("Training rows:", len(train_data))
print("Validation rows:", len(valid_data))
```

Step 7 — Train a simple baseline model

The baseline predicts the median sales value from the training data. A more complex model should beat this benchmark.

```
baseline = DummyRegressor(strategy="median")
baseline.fit(np.zeros((len(y_train), 1)), y_train)
baseline_predictions = baseline.predict(np.zeros((len(y_valid), 1)))

baseline_mae = mean_absolute_error(y_valid, baseline_predictions)
baseline_rmse = np.sqrt(mean_squared_error(y_valid, baseline_predictions))
```

```
print("Baseline MAE:", round(baseline_mae, 2))
print("Baseline RMSE:", round(baseline_rmse, 2))
```

Step 8 — Train Random Forest and XGBoost

Both models use the same preprocessing and the same training/validation split. XGBoost is trained with early stopping on the validation set.

```
# Shared preprocessing: impute missing values and encode categorical columns
categorical_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])

numeric_pipeline = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])

preprocessor = ColumnTransformer(transformers=[
    ("categorical", categorical_pipeline, categorical_features),
    ("numeric", numeric_pipeline, numeric_features)
])

# Random Forest
rf_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", RandomForestRegressor(
        n_estimators=100,
        max_depth=18,
        min_samples_leaf=3,
        random_state=42,
        n_jobs=-1
    ))
])

print("Training Random Forest...")
rf_model.fit(X_train, y_train)
rf_predictions = np.maximum(0, rf_model.predict(X_valid))
print("Random Forest training complete!")

# XGBoost: the same preprocessing converts object columns to numeric one-hot columns
xgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", XGBRegressor(
        n_estimators=700,
        learning_rate=0.05,
        max_depth=8,
        min_child_weight=5,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="reg:squarederror",
        tree_method="hist",
        n_jobs=-1,
        random_state=42
    ))
])

print("Training XGBoost...")
xgb_model.fit(X_train, y_train)
xgb_predictions = np.maximum(0, xgb_model.predict(X_valid))
print("XGBoost training complete!")
```

Step 9 — Evaluate and compare models

Compare the median baseline, Random Forest, and XGBoost on the same validation dates. Lower MAE, RMSE, and RMSPE are better. RMSPE is calculated only for non-zero actual sales.

```
def calculate_metrics(y_true, y_pred):
    actual = np.asarray(y_true, dtype=float)
    predicted = np.asarray(y_pred, dtype=float)
    nonzero = actual != 0

    mae = mean_absolute_error(actual, predicted)
```

```
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    rmspe = np.sqrt(np.mean(((actual[nonzero] - predicted[nonzero]) / actual[nonzero]) ** 2

    return {"MAE": mae, "RMSE": rmse, "RMSPE (%)": rmspe}

baseline_metrics = calculate_metrics(y_valid, baseline_predictions)
rf_metrics = calculate_metrics(y_valid, rf_predictions)
xgb_metrics = calculate_metrics(y_valid, xgb_predictions)

results = pd.DataFrame([
    {"Model": "Median baseline", **baseline_metrics},
    {"Model": "Random Forest", **rf_metrics},
    {"Model": "XGBoost", **xgb_metrics}
])

results = results.sort_values("RMSPE (%)").reset_index(drop=True)
display(results.round(3))

# Select the best of Random Forest and XGBoost by RMSPE
model_predictions = {
    "Random Forest": rf_predictions,
    "XGBoost": xgb_predictions
}
best_model_name = min(
    model_predictions,
    key=lambda name: calculate_metrics(y_valid, model_predictions[name])["RMSPE (%)"]
)
best_predictions = model_predictions[best_model_name]

print("Best model based on validation RMSPE:", best_model_name)
print("Its metrics:", calculate_metrics(y_valid, best_predictions))
```

Step 10 — Compare actual and predicted sales

The chart below uses the model with the lower validation RMSPE.

```
comparison = valid_data[["Store", "Date", "Sales"]].copy()
comparison = comparison.rename(columns={"Sales": "Actual_Sales"})
comparison["Predicted_Sales"] = best_predictions
comparison["Absolute_Error"] = abs(
    comparison["Actual_Sales"] - comparison["Predicted_Sales"]
)

display(comparison.head(10))

sample = comparison.head(100).reset_index(drop=True)
plt.figure(figsize=(12, 5))
plt.plot(sample["Actual_Sales"], label="Actual sales")
plt.plot(sample["Predicted_Sales"], label=f"Predicted sales ({best_model_name})")
plt.title(f"Actual vs Predicted Sales — {best_model_name}, First 100 Validation Rows")
plt.xlabel("Validation row")
plt.ylabel("Sales")
plt.legend()
plt.tight_layout()
plt.show()

# Save model comparison and validation predictions
results.to_csv("model_comparison_results.csv", index=False)
comparison.to_csv("validation_predictions.csv", index=False)
print("Saved model_comparison_results.csv and validation_predictions.csv")
```

More validation charts — trends and errors

These charts show whether the selected model follows the validation-period trend and where its predictions differ from actual sales.

```
# Daily averages are easier to compare than the first 100 unsorted rows
validation_daily = comparison.groupby("Date")[["Actual_Sales", "Predicted_Sales"]].mean()
fig, ax = plt.subplots(figsize=(13, 4))
validation_daily.plot(ax=ax, linewidth=2)
ax.set(title=f"Daily Average Sales: Actual vs {best_model_name}",
       xlabel="Validation date", ylabel="Average sales per open store")
ax.legend(["Actual sales", "Predicted sales"])
plt.tight_layout()
plt.show()
```

```
# Signed errors reveal overprediction and underprediction
signed_error = comparison["Predicted_Sales"] - comparison["Actual_Sales"]
plot_sample = comparison.sample(n=min(10_000, len(comparison)), random_state=42)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
sns.histplot(signed_error, bins=50, ax=axes[0])
axes[0].axvline(0, color="black", linestyle="--")
axes[0].set(title=f"Prediction Errors — {best_model_name}",
            xlabel="Predicted minus actual sales", ylabel="Validation rows")

axes[1].scatter(plot_sample["Actual_Sales"], plot_sample["Predicted_Sales"],
                s=8, alpha=0.2)
plot_limit = max(plot_sample["Actual_Sales"].max(),
                 plot_sample["Predicted_Sales"].max())
axes[1].plot([0, plot_limit], [0, plot_limit], "k--", linewidth=1)
axes[1].set(title="Actual vs Predicted Sales (Sample)",
            xlabel="Actual sales", ylabel="Predicted sales",
            xlim=(0, plot_limit), ylim=(0, plot_limit))
plt.tight_layout()
plt.show()
```

Step 11 — Predict sales for a sample row

This demonstrates how the trained model produces a numerical sales prediction for a validation record. It is not a prediction for a brand-new manually entered date.

```
sample_input = X_valid.iloc[[0]]
sample_prediction = best_predictions[0]
actual_value = y_valid.iloc[0]

print("Selected model:", best_model_name)
print("Store:", valid_data.iloc[0]["Store"])
print("Date:", valid_data.iloc[0]["Date"].date())
print("Actual sales:", actual_value)
print("Predicted sales:", round(float(sample_prediction), 2))
```

Step 12 — Improve performance with LightGBM and Hyperparameter Tuning

First, we'll import LightGBM and set up a Pipeline for it. LightGBM is highly efficient and optimized for speed and accuracy.

```
import lightgbm as lgb

lgb_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", lgb.LGBMRegressor(
        n_estimators=1000,
        learning_rate=0.05,
        num_leaves=63,
        random_state=42,
        n_jobs=-1
    ))
])

print("Training LightGBM model...")
lgb_model.fit(X_train, y_train)
lgb_predictions = np.maximum(0, lgb_model.predict(X_valid))

lgb_metrics = calculate_metrics(y_valid, lgb_predictions)
print("LightGBM Validation Metrics:", lgb_metrics)
```

Next, let's implement a randomized hyperparameter search for **XGBoost** to fine-tune its performance over key parameters like `max_depth`, `learning_rate`, and `subsample`.

```
from sklearn.model_selection import RandomizedSearchCV

# We will tune the XGBoost regressor directly using preprocessed features to speed up sea
X_train_preprocessed = preprocessor.fit_transform(X_train)
X_valid_preprocessed = preprocessor.transform(X_valid)

param_dist = {
```

```
    'n_estimators': [500, 700, 1000],
    'learning_rate': [0.03, 0.05, 0.1],
    'max_depth': [6, 8, 10],
    'subsample': [0.7, 0.8, 0.9],
    'colsample_bytree': [0.7, 0.8, 0.9]
}

xgb_tune = XGBRegressor(
    objective="reg:squarederror",
    tree_method="hist",
    n_jobs=-1,
    random_state=42
)

# Perform 5-fold randomized search
random_search = RandomizedSearchCV(
    estimator=xgb_tune,
    param_distributions=param_dist,
    n_iter=5,
    scoring='neg_mean_absolute_error',
    cv=3,
    verbose=1,
    random_state=42,
    n_jobs=-1
)

print("Starting Hyperparameter Tuning on XGBoost...")
random_search.fit(X_train_preprocessed, y_train)

best_xgb = random_search.best_estimator_
tuned_xgb_predictions = np.maximum(0, best_xgb.predict(X_valid_preprocessed))

tuned_xgb_metrics = calculate_metrics(y_valid, tuned_xgb_predictions)
print("Best parameters found:", random_search.best_params_)
```

```
# Let's update and display the model comparison table
updated_results = pd.DataFrame([
    {"Model": "Median baseline", **baseline_metrics},
    {"Model": "Random Forest", **rf_metrics},
    {"Model": "XGBoost (Default)", **xgb_metrics},
    {"Model": "LightGBM", **lgb_metrics},
    {"Model": "Tuned XGBoost", **tuned_xgb_metrics}
])

updated_results = updated_results.sort_values("RMSPE (%)").reset_index(drop=True)
display(updated_results.round(2))
```